

Ledger Line

A dark factory that proves its work.

Four coding-agent seats turn a spec into a numbered ledger, build a payments app to it, and accept nothing until an independent prover and auditor sign the **same commit**.

4

Claude Code seats, generic mandates

1

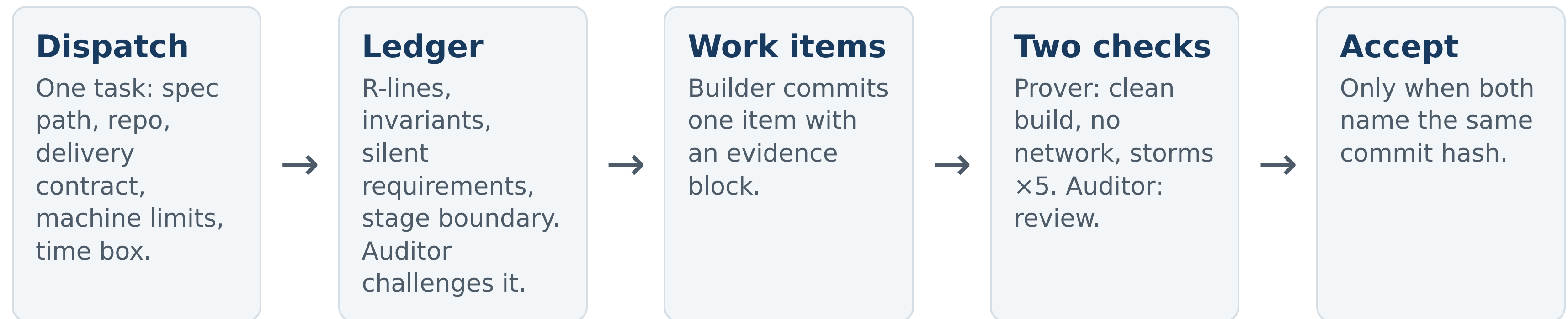
human message per stage: the dispatch

Stage 2

claimed, isolated mode:
147/147 + 35/35

Factory design: the spec drives, not the samples

The shipped checks cover ~79% of stage 1; the rest lives in the spec's prose, and money breaks under concurrency, retries and rounding. So every testable sentence becomes a ledger line *before* any code exists.



The mandates never name the track. Hand them a different spec and they work unchanged.

Four seats, each with a boundary

Seat	Owns	Never does
lead	the ledger, the plan, routing, acceptance, the final report	writes product code or tests
builder	source, Dockerfile, RUN.md, unit tests	accepts its own work; edits the prover's tests
prover	black-box tests derived from the ledger; every check run	reads product code to decide what to test; fixes code
auditor	ledger challenge; review of every revision	fixes code; rejects without a concrete defect

All four: Claude Code · `claude-opus-5-5` . Handoffs are self-contained (requirements pasted in full), end with an evidence block (revision · ledger lines · commands · result · not checked), and three rejections of one item force a re-plan.

How it catches bad work, in the judged run

STAGE 1 · API		STAGE 2 · BROWSER UI + HOLDS	
15:36	Ledger accepted; auditor added 12 points	17:34	Ledger carries S-23 forward; 13-point challenge
15:45	W2 rejected: 2 blocking huge-exponent number → HTTP 500; replay check rounded to 28 digits	17:50	X2 rejected: feed showed only 50 payments
15:50	Fix accepted at d0b69b4 (+18 min)	17:57	Hold expiry clamped with float seconds
16:07	Prover 440 / 441: S-23 reset too slow	17:59	Fixes accepted at 18849a9
		18:04	Reset 8.3 s on the laptop → cost halved, accepted 18:05
		18:17	Prover: no product failures; 35/35

What went wrong, and what we changed

- 16:10 **Stage 1 deadlock.** The prover's "one fail, S-23" reply was staged behind two background log watchers and never posted. Every seat waited; no human may break the tie. No stage-1 report.
- 17:32 **Fix, in the task, not the mandates:** the stage-2 dispatch adds a liveness rule. Nothing blocking attached to a turn, timed waits only, end every turn. Stage 2: 40 replies sent, 0 stuck.
- 18:18 **Stage 2 cut short from outside: the operator's helper agent stopped the seats, believing the run had finished.**

No code or messages were changed. The lead had just asked for one more prover round on the one-line acceptance revision, so the stage-2 report was never written.

Lessons: mandates should govern a handoff's *liveness*, not only its content. And a stopped seat looks exactly like a finished one: stop a run only on the coordinator's report.

What it reached, and what it cost

Stage 2 claimed in isolated mode (no network, 2 vCPU, 2 GiB): suite 1 **147/147**, suite 2 **35/35**; stage 3 fails, as it should. `stage-1/` still claims stage 1.

Acceptance: **18 min 38 s** (stage 1) and **27 min 38 s** (stage 2) after each dispatch.
Rejections that changed the work: 1, then 3.

Commits: lead 8 · builder 14 · prover 14 · auditor reviews only.

Room events: builder 535 · prover 508 · lead 369 · auditor 283.

Known limit: S-23 depends on the CPU.
1000-user reset: stage 2 1.8 s on a desktop, 6-8 s on a laptop, vs a 10 s limit.

≈ **\$15.06** model spend for stage 1, 24.4 M tokens (list-price estimate; stage 2 not attributed by Band)

prover \$6.33

builder \$4.37

aud \$2.38

lead

lead \$1.98 · rehearsal room ≈ \$4.55

Hand these four mandates any spec.
github.com/jphein/ledger-line